

Estrutura de Repetição

WHILE • FOR • BREAK / CONTINUE

Aprender a repetir blocos de código com elegância: laços controlados por condição, percurso de sequências e controle fino do fluxo.

🎓 Ciência da Computação e Sistemas de Informação — 2º Semestre

📖 Disciplina: Introdução à Programação Estruturada em Python

👥 Professores: Luciano, Rafael e Turatti



Roteiro de hoje

Cinco paradas entre você e o domínio dos laços em Python.



01 Por que repetir?

O problema das tarefas repetitivas e a ideia de automação dentro do programa.

02 WHILE

Repita **ENQUANTO** a condição for verdadeira: contador, acumulador e validação.

03 FOR

Repita **PARA CADA** item: `range()`, strings e listas com elegância.

04 BREAK & CONTINUE

Controle fino: encerrar o laço na hora ou pular uma iteração.

05 Prática

Erros comuns, boas práticas e exercícios para fixar.

Por que precisamos repetir?

SEM LAÇO

Imprimir Olá, Python! cem vezes dá cem linhas de print. Copiar e colar não escala: é cansativo, difícil de manter e qualquer ajuste exige corrigir cem lugares.

```
print('Olá, Python!')
print('Olá, Python!')
# ... e mais 98 linhas idênticas
```

COM LAÇO

```
for i in range(100):
    print('Olá, Python!')
```

Duas linhas. Zero duplicação.

No dia a dia da programação, quase tudo se repete:

Processar as notas de todos os alunos

Validar a senha até acertar

Somar as vendas de cada dia do mês

Laços (loops) executam um mesmo bloco várias vezes: é a automação vivendo dentro do seu programa.



WHILE

Repita ENQUANTO a condição for verdadeira.

O laço das situações em que você não sabe de antemão quantas repetições serão necessárias: validar uma senha, esperar um evento, rodar um menu.

```
while condicao: # True → repete
```

01

WHILE: anatomia do laço

```
anatomia.py

while condicao:
    # corpo do laço
    # repetido enquanto a condição for True
```



A condição precisa se tornar **False** em algum momento — senão o laço nunca termina.

1

Palavra-chave **while**

todo laço de condição começa por ela

2

Condição booleana

avaliada **ANTES** de cada iteração; só pode ser True ou False

3

Dois-pontos :

obrigatórios no fim da linha

4

Corpo indentado

4 espaços: tudo que está indentado repete

Testa a condição

→ **True:** executa o corpo e **VOLTA** ao teste

→ **False:** sai do laço e segue o programa

WHILE na prática: o contador

contador.py

```
1 contador = 1          # (1) inicialização
2 while contador ≤ 5:    # (2) condição de parada
    print(contador)
3     contador += 1      # (3) atualização
```

Vez	contador antes	imprime	depois
1	1	1	2
2	2	2	3
3	3	3	4
4	4	4	5
5	5	5	6
fim	6	—	6 ≤ 5 é False → para

SAÍDA

1 2 3 4 5



Inicializar → Testar → Atualizar. Esqueceu o passo (3)? Laço infinito.

Acumulando valores durante o laço

```
acumulador.py

soma = 0          # acumulador começa em zero
numero = 1
while numero <= 10:
    soma += numero # acumula o valor atual
    numero += 1
print(f'Soma de 1 a 10 = {soma}')
```

123 CONTADOR

conta iterações: soma
sempre +1 (contador += 1).
Responde: **quantas vezes?**

🏠 ACUMULADOR

soma valores que variam:
soma += numero. Responde:
quanto no total?

SAÍDA

Soma de 1 a 10 = **55**



O acumulador também precisa de valor inicial: **0** para somas, **1** para produtos.

O uso clássico do WHILE: validar entrada

senha.py

```
senha = input('Digite a senha: ')
while senha != 'python123':
    print('Senha incorreta! Tente de novo.')
    senha = input('Digite a senha: ')
print('Acesso liberado!')
```

POR QUE WHILE?

Ninguém sabe quantas tentativas o usuário vai precisar. O laço roda até um **EVENTO** acontecer: a senha acertar. **FOR** não se encaixa: não há contagem conhecida.

PADRÃO SENTINELA

Um valor especial sinaliza o fim da repetição — digitar **0** para sair, **-1** para encerrar a leitura de notas, linha vazia para terminar. É o **sinal de parada** do laço.

⚠️ Atenção: o laço infinito

bug.py

```
x = 1
while x ≤ 5:
    print(x)
    # faltou: x += 1
```

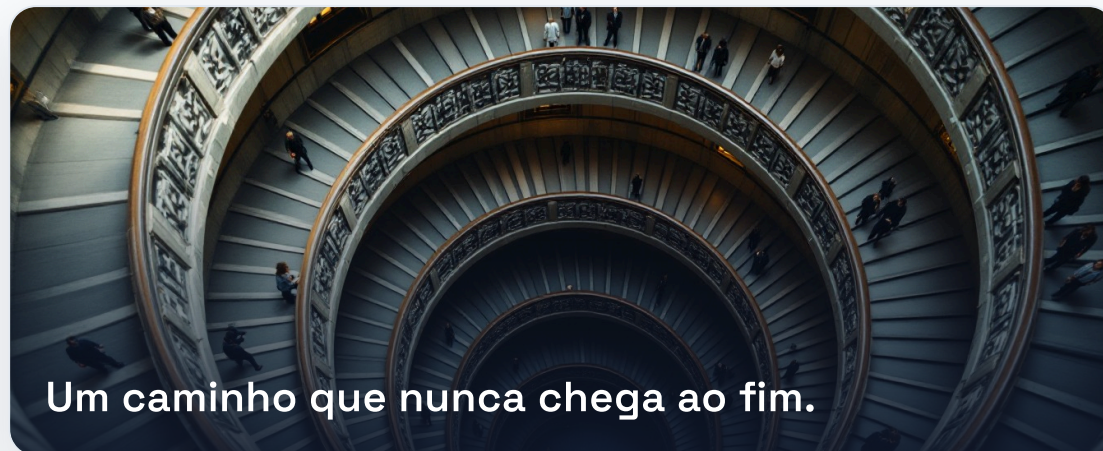
O QUE ACONTECE

x nunca muda, então $x \leq 5$ é sempre **True**. O programa imprime **1** sem parar — e trava.

COMO INTERROMPER

Ctrl+C no terminal ·

botão  **Stop** (quadrado) do editor/IDE.



Um caminho que nunca chega ao fim.

CHECKLIST ANTI-TRAVAMENTO

- 1 A variável da condição foi inicializada antes do **while**?
- 2 Ela é atualizada **DENTRO** do corpo do laço?
- 3 A atualização a aproxima da condição virar **False**?

FOR

Repita PARA CADA item de uma sequência.

O laço mais usado do Python: percorre listas, strings e intervalos de números com código curto e legível.

01 WHILE · 02 FOR · 03 BREAK & CONTINUE

```
for item in sequencia: # repete por item
```

FOR: sintaxe e iteráveis

forma_geral.py

```
for variavel in sequencia:  
    # corpo do laço
```

frutas.py

```
for fruta in ['maçã', 'uva', 'pera']:  
    print(fruta)
```



variavel — assume cada valor da sequência, um por vez



in — liga a variável à sequência



iterável — qualquer coleção de itens: **range**, string, lista

SAÍDA

maçã

uva

pera

FOR + range(): três formas de contar

`range(stop)`

`range(5)` gera

0 1 2 3 4

Começa em **0** e vai até **stop - 1**.

`range(start, stop)`

`range(1, 6)` gera

1 2 3 4 5

Começa em **start** e vai até **stop - 1**.

`range(start, stop, step)`

`range(0, 10, 2)` gera

0 2 4 6 8

Anda de **step** em **step**.



Armadilha clássica: o valor de **stop** **NUNCA** entra na contagem. `range(5)` tem 5 números... e termina no **4**.

BÔNUS

Contagem regressiva: `range(5, 0, -1)` gera **5 4 3 2 1**.

FOR: percorrendo strings e listas

■ Percorrendo uma string

```
nome = 'Python'
for letra in nome:
    print(letra)
```

SAÍDA

P • y • t • h • o • n

uma letra por linha

■ Percorrendo uma lista + acumulador

```
notas = [7.5, 8.0, 6.5]
soma = 0
for nota in notas:
    soma += nota
media = soma / len(notas)
print(f'Média = {media:.2f}')
```

SAÍDA

Média = 7.33



Precisa do índice da posição? Use a dupla: `for i in range(len(notas)):`

FOR vs WHILE: qual usar?

↺ FOR

quando você sabe **QUANTAS** vezes

- ✓ Número de repetições conhecido (10 vezes, 100 voltas)
- ✓ Percorrer sequências: listas, strings, range
- ✓ Tabuadas, contagens, processar coleções

∞ WHILE

quando você sabe **ATÉ QUANDO**

- ✓ Repetir até um evento: senha correta, usuário digita 0
- ✓ Repetições de quantidade desconhecida
- ✓ Menus e jogos que rodam até o usuário sair

Sabe QUANTAS vezes? → FOR. Sabe ATÉ QUANDO? → WHILE.

Em Python, todo FOR pode ser reescrito com WHILE — o FOR vence em legibilidade e concisão.

BREAK & CONTINUE

Controle fino do fluxo do laço.

Duas palavras-chave que decidem, a cada iteração, se o laço termina agora ou pula direto para a próxima volta.

`break` → encerra o laço inteiro

·

`continue` → pula só esta volta

BREAK: encerre o laço agora

busca_break.py

```
for numero in range(1, 11):  
    if numero == 6:  
        print('Achei o 6! Parando por aqui.')  
        break  
    print(numero)
```

SAÍDA

1 2 3 4 5

Achei o 6! Parando por aqui.

O QUE FAZ

Interrompe o laço **INTEIRO** na hora. O programa segue na primeira linha depois do laço. O **6** não é impresso: o **break** dispara antes do print.

USOS TÍPICOS



Achar um item numa lista — achou, pare



Sair de um **while True** quando o evento acontece



Encerrar um menu quando o usuário escolhe sair

CONTINUE: pule esta iteração

● ● ● pares.py

```
for numero in range(1, 8):  
    if numero % 2 == 0:    # é par?  
        continue        # pula o print  
    print(numero)
```

📄 SAÍDA

1 3 5 7

► O QUE FAZ

Abandona **SÓ a volta atual**: tudo abaixo do `continue` é ignorado e o laço segue para o próximo item. Os pares **2, 4 e 6** nunca chegam ao print.

DUELO RÁPIDO

◼ BREAK

adeus, laço! — encerra tudo de uma vez.

VS

►► CONTINUE

essa não conta, próxima! — pula e continua.

Juntando tudo: busca em lista

busca.py

```
alunos = ['Ana', 'Bruno', 'Carla', 'Diego']
achou = False

for aluno in alunos:
    print(f'Verificando {aluno}...')
    if aluno == 'Carla':
        achou = True
        break          # não precisa olhar o resto

if achou:
    print('Encontrado!')
else:
    print('Não está na lista')
```

SAÍDA

```
Verificando Ana...
Verificando Bruno...
Verificando Carla...
Encontrado!
```



INSIGHT

Achou? Pare. Não faz sentido continuar procurando depois de encontrar — o **break** encerra a busca no momento exato.

POR QUE ECONOMIZA

4

SEM BREAK

VS

3

COM BREAK

Sem break: sempre **4 verificações**. Com break: paramos na **3ª**. Em listas de milhares de itens, a economia é enorme.

Mini-projeto: Jogo de Adivinhação

🎮 MINI-PROJETO while True + break

adivinhacao.py

```
secreto = 7
tentativas = 0

while True:
    palpite = int(input('Palpite (1-10): '))
    tentativas += 1
    if palpite == secreto:
        print(f'Acertou em {tentativas} tentativas!')
        break
    elif palpite < secreto:
        print('É MAIOR! Tente de novo.')
    else:
        print('É MENOR! Tente de novo.')
```

1

while True: laço infinito proposital — roda até o **break** autorizar a saída.

2

break: a porta de saída quando o evento acontece (acertar o número).

3

tentativas: acumulador que conta cada volta do laço.



Esse é o padrão de menus, jogos e sistemas reais: **rode sempre, saia quando fizer sentido.**

Erros comuns (e como escapar)



Laço infinito

Faltou atualizar a variável de controle.

Correção: atualize dentro do corpo.

+1

Off-by-one

`range(1, 5)` vai só até 4. O stop é exclusivo — conte nos dedos antes de codar.

`=x`

= no lugar de ==

`while x = 5:` é `SyntaxError`. Atribuição é `=`; comparação é `==`.



Indentação

Linha entrou no laço sem querer (ou saiu). Em Python, indentação é sintaxe: 4 espaços.



Condição impossível

`while contador > 0` com contador crescendo: nunca para. Teste o sinal da lógica.



Editar a lista ao iterar

Remover itens durante o `for` pula elementos. Itere sobre uma cópia: `for x in lista[:]`:

 KIT DE DEBUG



`print()` de inspeção dentro do laço



Teste primeiro com poucas iterações



Tabela de execução no papel

Boas práticas de laços

1



Escolha o laço certo

FOR para percorrer sequências; **WHILE** para condições e eventos.

2



Nomeie com significado

`nota`, `tentativa`, `media`. `i` e `j` só em contadores simples.

3



Inicialize fora do laço

Acumulador de soma começa em **0**; de produto, em **1**.

4



break/continue com moderação

Ótimos em buscas; em excesso, tornam o fluxo difícil de seguir.

5



Cuidado com laços aninhados

Dois laços de **1000** executam **1.000.000** de passos. Vale a pena?

6



Teste os limites

Primeira volta, última volta e o caso de zero repetições.

Mãos ao código: exercícios

1

FOR

Tabuada

Leia um número e mostre a tabuada de 1 a 10.

2

FOR + range

Pares espertos

Imprima os pares de 0 a 50 usando `range()` com step.

3

WHILE

Porta blindada

Peça a senha até o usuário digitar `python123`.

4

WHILE + acumulador

Média da turma

Leia notas até digitar `-1`; mostre a média e o total de alunos.

5

BREAK

Número primo

Teste divisores e pare no primeiro divisor encontrado.

6

CONTINUE

Filtro múltiplos

De 1 a 20, imprima só quem **NÃO** é múltiplo de 3.



BÔNUS

CAIXA ELETRÔNICO — menu com `while True` + `break`: depositar, sacar, ver saldo, sair.

O que levamos hoje

WHILE — repita enquanto a condição é **True**: rei da validação e dos menus.

FOR — repita para cada item: perfeito com **range()**, listas e strings.

BREAK encerra o laço na hora; **CONTINUE** pula para a próxima volta.

Contador e acumulador: inicializar, testar, atualizar — e nada de laço infinito.



Levem os exercícios!

Dúvidas? Tragam para a **monitoria** ou para a **próxima aula**.